

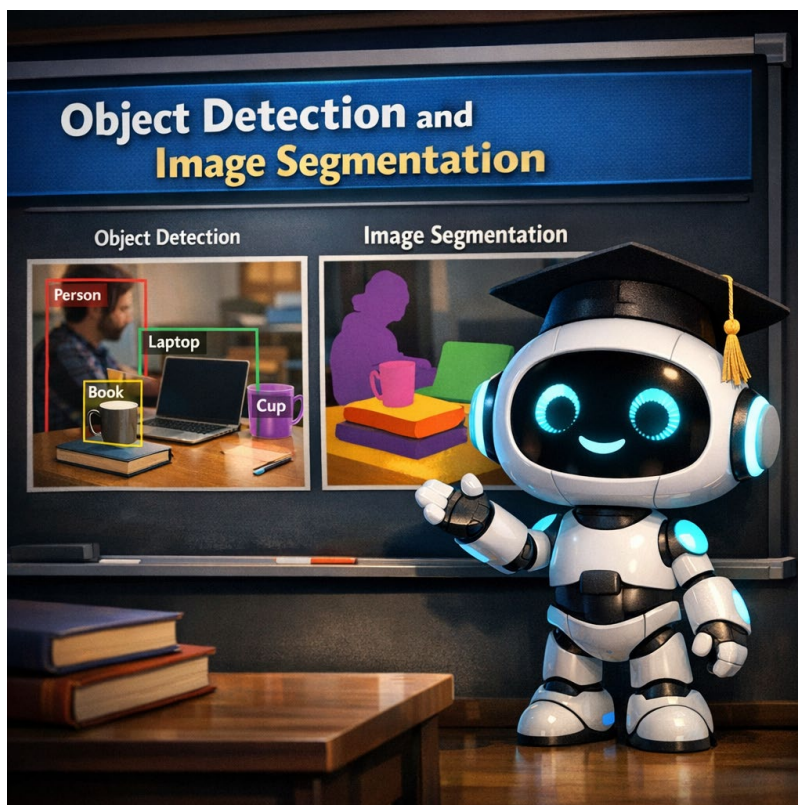
ITAI 1378

Computer Vision for Artificial Intelligence

Module 06

Object Detection and Image Segmentation

From 'is there a cat' to 'where is the cat' to 'which pixels are the cat'.



Author: Patricia McManus

Collaborator: Claude (Anthropic)

Welcome to Module 06

This is the module that takes the CNN backbone you built in Module 05 and points it at two of the most commercially important problems in computer vision: object detection (finding where each thing is in an image) and image segmentation (drawing pixel level outlines of every thing). Both tasks build directly on the convolutional and transfer learning foundation from Module 05. The CNN backbone is the same; what changes is what comes after the backbone.

You will leave this module able to do three things. First, you will understand how detection works: bounding boxes, anchor boxes, non maximum suppression, and the metrics (IoU, mAP, precision, recall) used to evaluate detectors. Second, you will know the modern detector families: the YOLO line for fast real time work, the DETR line for transformer based approaches, and GroundingDINO for open vocabulary detection. Third, you will understand image segmentation at every level: the three types (semantic, instance, panoptic), the workhorse models (U-Net, Mask R-CNN), the metrics that measure them (mIoU, Dice, pixel accuracy), and the foundation model approach (Segment Anything) that changed segmentation from a custom training task into a callable service.

This module is also where the midterm project (Version 1) is introduced. By the end of M06 you will have all the techniques you need for the first half of your semester project. Like every booklet in this course, this is an evergreen reference. Specific code examples and library versions will evolve. The concepts (what detection adds, why segmentation is harder, how foundation models reshape practice) will not.

KEY TAKEAWAY | **What this module will give you**

You will be able to pick the right detector and segmentation approach for any CV problem, read and understand a detection or segmentation paper, train a YOLO detector on a custom dataset using transfer learning from Module 05, use Segment Anything as a callable service without needing custom training, and avoid the common pitfalls that trip up beginners. This is the workforce baseline for detection and segmentation in 2026.

Part 01 | From Classification to Localization

Computer vision tasks live on a spectrum of how much information they extract from an image. Classification, which you have been doing in Module 05, sits at the cheap end. Segmentation, which you will reach by the end of this module, sits at the rich end. Detection lives in between. This part frames the three tasks side by side so the rest of the module makes sense in context.

The Three Tasks Compared

Classification answers the question 'what is in this image?' with a single label for the whole image. A photo gets labeled 'cat' or 'dog'. The output is one number per class: a probability distribution over the possible categories.

Detection answers a richer question: 'what is in this image, and where is each thing?' For every object the model sees, it returns a bounding box (a rectangle) and a class label. A photo of a kitchen might return: 'fork at [120, 80, 180, 200]', 'plate at [250, 100, 400, 280]', 'cup at [410, 90, 460, 200]', and so on. The output is a variable length list of labeled boxes.

Segmentation goes one level deeper: 'which specific pixels are which?' For every pixel in the image, the model assigns a label. The output is the same size as the input image: a mask in which every pixel has a class. Instead of a box around a cat, you get the actual silhouette of the cat.



INDUSTRY PRACTICE | Why these three tasks dominate

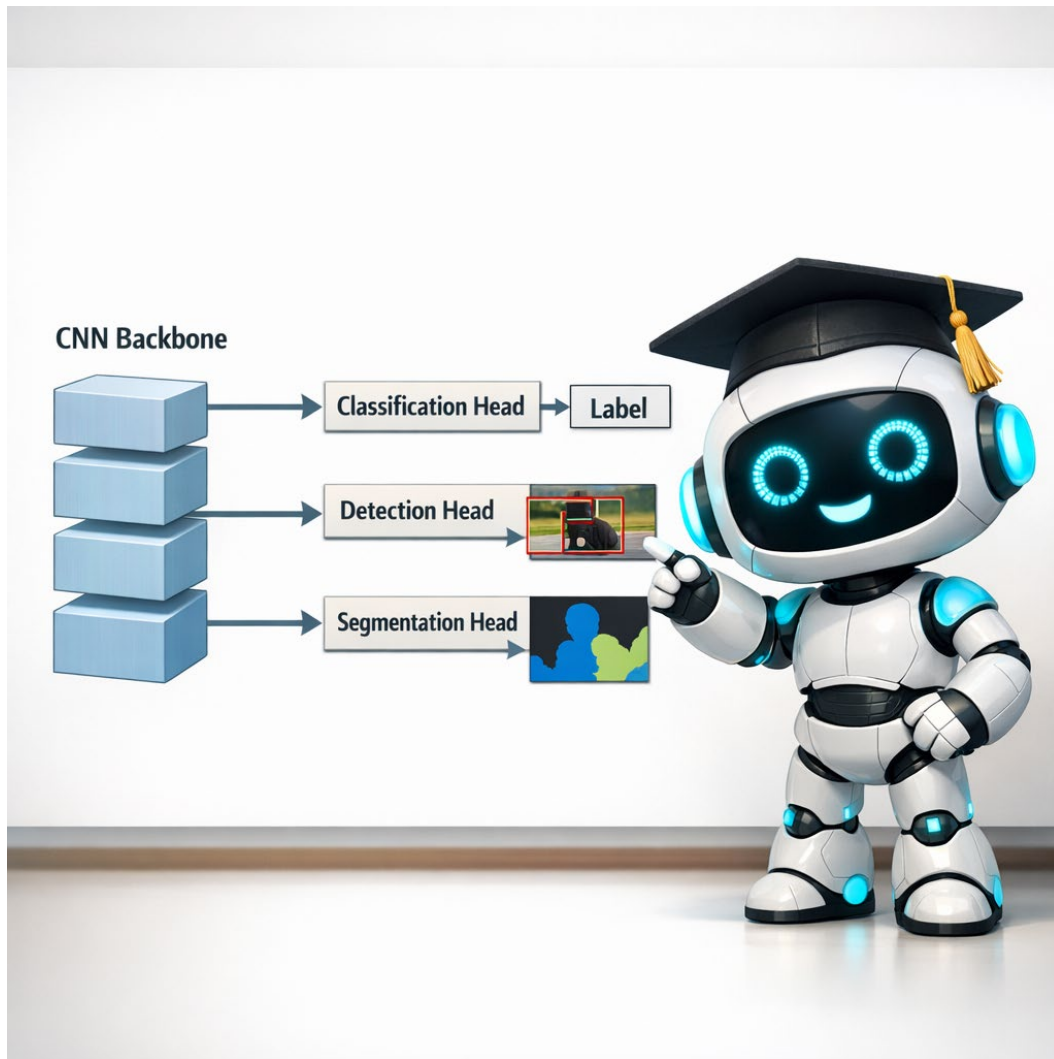
Almost every commercial CV system in 2026 falls into one of these three buckets. Classification: spam filtering, content moderation, product categorization. Detection: autonomous driving, retail analytics, manufacturing inspection, license plate readers. Segmentation: medical imaging (tumor boundaries), autonomous driving (road and lane masks), photo editing (background removal), robotics (graspable surfaces). Knowing which bucket your problem belongs in is half the work of designing a CV system.

The Shared Backbone

Here is the good news. All three tasks (classification, detection, segmentation) share the same convolutional backbone. The CNN you learned about in Module 05 produces a stack of feature maps that capture the visual content of the image. Classification adds a small fully connected head on top to produce a single label. Detection adds a more elaborate head that produces boxes and labels. Segmentation adds a head that produces a pixel level mask. The backbone is shared; the heads differ.

This sharing is why transfer learning (also from M05) is so powerful in detection and segmentation. You can start with a CNN backbone pretrained on ImageNet classification, swap out the classification head for a detection or segmentation head, and fine tune on your specific task. Almost every modern detector and segmenter is built this way.

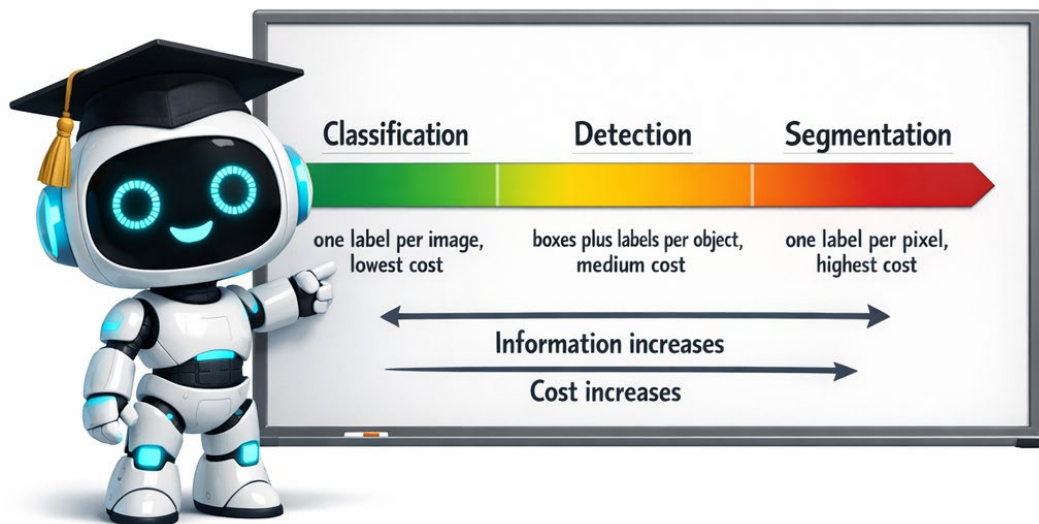




The Information Versus Cost Tradeoff

More information requires more from the model and more from you. Classification needs the smallest dataset (a few hundred labeled images per class can work), trains the fastest, and produces the simplest output. Detection needs bounding box labels (more expensive than class labels) and a more complex output structure. Segmentation needs pixel level masks (the most expensive form of annotation, often hours per image for skilled annotators) and produces the richest output.

Pick the simplest task that meets your needs. If you only need to know whether a defect is present, classification is enough. If you need to know where each defect is, you need detection. If you need to measure the area or shape of each defect, you need segmentation. Engineering judgment is choosing the cheapest task that gives you the answer you need.



⚠ MISCONCEPTION ALERT | Segmentation is not always better

It is tempting to think 'segmentation gives the most information, so it must be the best choice'. Not always. Segmentation requires pixel level labels, which can cost ten times more to produce than bounding boxes. Segmentation models are slower at inference, harder to deploy on edge devices, and harder to debug. If a bounding box answers the business question, do not pay for a mask. Right tool, right job.

🎯 KEY TAKEAWAY | Part 01 in one paragraph

Three CV tasks on a spectrum of information richness: classification (one label per image), detection (boxes plus labels per object), segmentation (one label per pixel). All three share the same CNN backbone from Module 05; the differences are in the heads on top. More information costs more in labels, compute, and complexity. Pick the simplest task that answers your business question.

✓ **KNOWLEDGE CHECK** | Quick check on three tasks

1. Describe the output format of each task: classification, detection, segmentation.
2. A factory wants to count how many defective parts pass on a conveyor belt each minute. Which task best fits? Why not the others?
3. Why do almost all modern detectors and segmenters use the same kind of CNN backbone as classifiers?

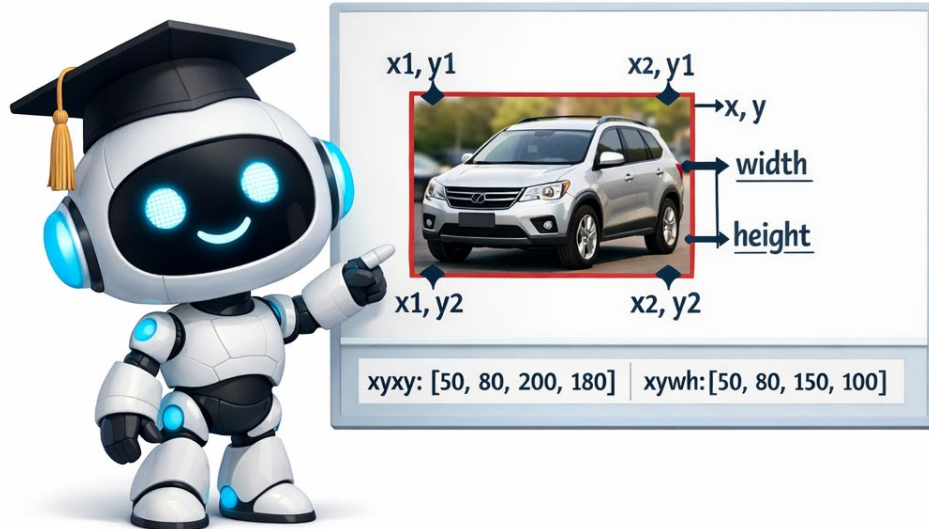
Part 02 | Detection Mechanics: Boxes, Anchors, and NMS

Now that you know what detection is, this part explains how it works. Three core mechanisms make detection possible: the bounding box format itself, the anchor or query system that proposes candidate locations, and the non maximum suppression step that cleans up duplicate predictions. Every modern detector uses some version of these three. This part also walks the R-CNN lineage, the family of two stage detectors that produced Mask R-CNN (which you will meet in Part 05).

The Bounding Box

A bounding box is just four numbers that describe a rectangle. There are two common conventions. The `xyxy` convention stores the top left corner and the bottom right corner: `x1, y1, x2, y2`. The `xywh` convention stores the top left corner plus the width and height: `x, y, w, h`. Different frameworks use different conventions, so always check the documentation when reading detector output.

In addition to the four box numbers, every detection includes two more pieces of information: a class label (what kind of object is this) and a confidence score (how certain the model is). A complete detection looks like: 'car at [120, 80, 450, 260] with confidence 0.93'. The confidence is a number between zero and one, with higher meaning more certain.



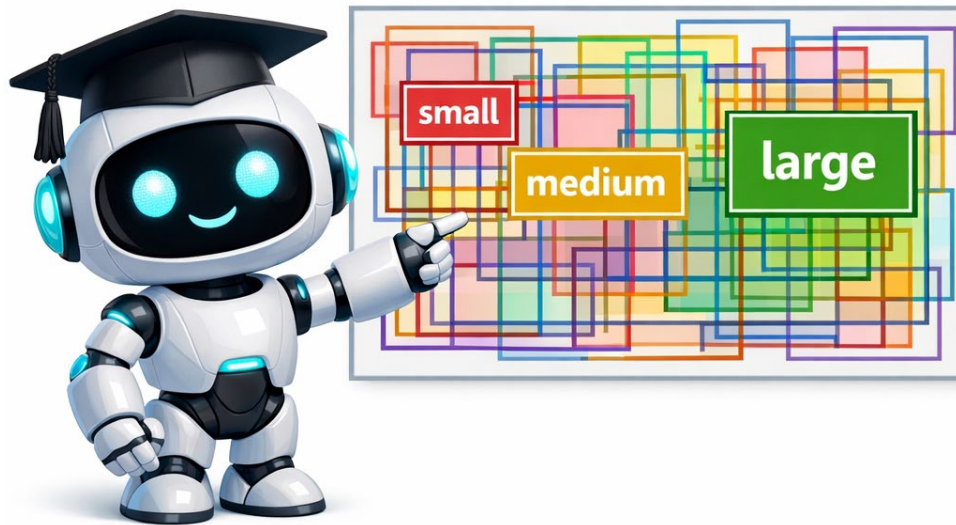
DEEPER LOOK | Coordinates in plain English

Think of the image like a piece of graph paper. The top left corner of the image is (0, 0). The bottom right is (image_width, image_height). A bounding box's top left corner is the point where the rectangle starts; its bottom right corner is where it ends. That is it. No special math, just four numbers per box. The frameworks handle drawing the rectangles on screen; you only need to remember the format.

Anchor Boxes: Candidate Generation

A natural question is: how does the detector decide where to draw boxes? The image is full of empty space and could contain objects of any size. Modern detectors solve this with anchor boxes (sometimes called priors or default boxes). Anchors are a fixed set of preset box templates placed at every position on a grid across the image. For each anchor, the model asks two questions: does this anchor contain an object, and if yes, how should the box be adjusted to fit the object better?

The anchors come in different sizes and aspect ratios so that small thin objects and large square objects can both be detected. A typical detector might have nine anchors at every grid position: three sizes (small, medium, large) times three aspect ratios (tall, square, wide). With a grid of, say, twenty by twenty positions, that gives 3,600 anchor boxes the model is examining. The model predicts, for each anchor, whether an object is present and how the box should shift to match.



Newer Approach: Anchor Free Detectors

Some modern detectors (DETR and RT-DETR, covered in Part 03) skip anchors entirely. Instead of preset templates, they have a small set of learnable query vectors. Each query directly predicts a single object location. There is no grid of anchors, no hand designed templates. The transformer architecture (Module 08) handles the assignment of queries to objects internally. This is cleaner conceptually but historically was slower; recent versions have caught up.

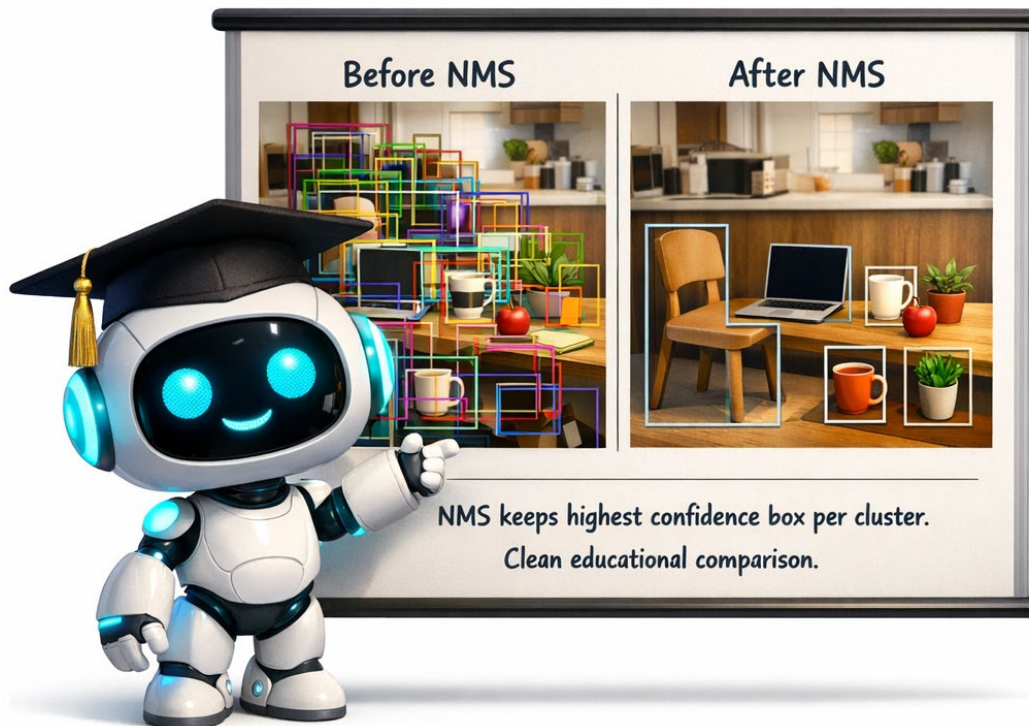
In 2026 you will encounter both anchor based detectors (the YOLO family up through some recent versions) and anchor free detectors (DETR, RT-DETR, and newer YOLO variants that have moved away from anchors). Knowing both styles is part of the workforce baseline.

Non Maximum Suppression: Killing Duplicates

Detectors produce many predictions, often overlapping. If three anchors near a traffic light all fire, you end up with three bounding boxes around the same traffic light. The cleanup step that solves this is called non maximum suppression, or NMS. The algorithm is simple. Sort all the predictions by confidence. Take the highest confidence box. Look at all other boxes that overlap with it significantly. Suppress (discard) all of them. Repeat with the next highest confidence box. Continue until no boxes remain.

The overlap threshold is usually defined by intersection over union (IoU), which Part 03 covers. A common threshold is 0.5, meaning boxes that overlap more than 50 percent of their combined

area are considered duplicates. NMS is usually built into the detector library you use; you do not need to write it yourself, but you should understand what it does.



One Stage Versus Two Stage Detectors

Detectors come in two architectural families. Two stage detectors (like Faster R-CNN and Mask R-CNN) do detection in two steps. First, a region proposal network generates a list of candidate regions that might contain objects. Second, a classifier examines each candidate region and decides what is there. Two stage detectors are usually more accurate but slower because of the two pass nature.

One stage detectors (like the YOLO family, SSD, and the modern transformer based detectors) do everything in a single forward pass. The network looks at the image once and produces all the boxes and labels at the same time. One stage detectors are faster, which is essential for real time applications. The gap in accuracy between one stage and two stage has narrowed in

recent years; modern one stage detectors are nearly as accurate as two stage ones while being much faster.

INDUSTRY PRACTICE | In 2026, one stage wins production

Almost every production deployment in 2026 uses a one stage detector. The reasons: real time inference is needed for live video, autonomous systems, and interactive applications, and modern one stage detectors are accurate enough for almost any use case. Two stage detectors still appear in research and in domains where every percentage point of accuracy matters more than speed (some medical imaging), but as a default, reach for one stage.

The R-CNN Lineage

The two stage detector family has a clear genealogy worth knowing. Four versions, each fixing a problem in the previous one. Knowing the lineage lets you read any detection or segmentation paper from the last decade and understand where the techniques came from.

R-CNN (2014). The first deep learning detector to make a serious impact. The approach: use a classical computer vision algorithm to propose 2000 candidate regions per image, then run a CNN on each region separately to classify it. This was breakthrough accuracy at the time, but training and inference were both extremely slow because the CNN had to process each region from scratch.

Fast R-CNN (2015). Solves the slowness. Instead of running the CNN on each region from scratch, run the CNN once over the whole image, then look up the features for each candidate region from the resulting feature map. Much faster training and inference, same overall accuracy idea.

Faster R-CNN (2015). Solves a remaining bottleneck: the region proposal step was still using a slow classical algorithm. Faster R-CNN replaces it with a learned subnetwork called the Region Proposal Network (RPN). The RPN looks at the same CNN features and proposes candidate regions directly. End to end deep learning, no hand designed proposal step. This is the architecture most production two stage detectors descend from.

Mask R-CNN (2017). Takes Faster R-CNN and adds one more component: a mask prediction head that produces pixel level outlines in addition to the bounding boxes and class labels. The first three components (backbone, RPN, classification head) are inherited unchanged from Faster R-CNN. The fourth component (mask head) is the only new piece. This is why Mask R-CNN is described as instance segmentation built on a detector. Part 05 covers Mask R-CNN in detail.



DEEPER LOOK | Why this lineage still matters in 2026

Even though most production work now uses one stage detectors (YOLO, RT-DETR), the R-CNN lineage is still alive in research and in domains where accuracy matters more than speed. More importantly, knowing the lineage gives you the vocabulary to read any paper from the last decade. When a paper mentions 'building on Faster R-CNN' or 'using an RPN', you now know exactly what it means and where it came from. Detection and segmentation history is short enough to actually know.

KEY TAKEAWAY | Part 02 in one paragraph

Three mechanisms power modern detection. Bounding boxes are just four numbers plus a class and a confidence. Anchor boxes (or learnable queries in newer detectors) propose candidate locations. Non maximum suppression cleans up duplicate predictions. One stage detectors dominate production; two stage detectors still appear in research and accuracy critical domains. The R-CNN lineage (R-CNN, Fast R-CNN, Faster R-CNN, Mask R-CNN) is the genealogy of the two stage family.

✓ **KNOWLEDGE CHECK** | **Quick check on mechanics**

1. What are the two common conventions for representing a bounding box, and what are the four numbers in each?
2. Why do detectors need anchor boxes (or learnable queries)? What problem do they solve?
3. What does non maximum suppression do, and what would the detector output look like without it?
4. Walk the four steps of the R-CNN lineage and describe what each step fixed.

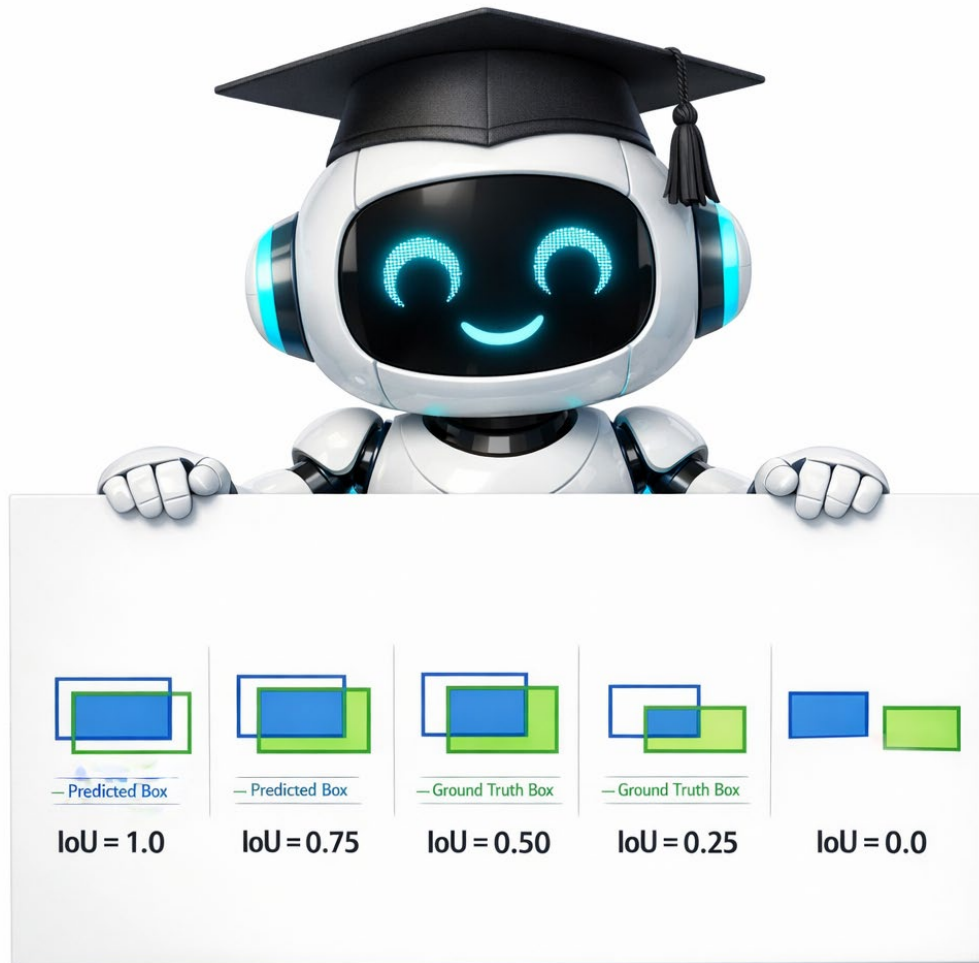
Part 03 | Detection Metrics and Modern Families

This part covers two things. First, the metrics used to evaluate detectors (IoU, precision, recall, mAP) so you can measure and compare them. Second, the modern detector families that dominate production: the YOLO line, the DETR and RT-DETR transformer detectors, and the open vocabulary GroundingDINO. By the end of this part you will be able to read a detection paper and understand the leaderboard.

Intersection Over Union (IoU)

The fundamental detection metric is intersection over union, or IoU. It measures how well a predicted box matches a ground truth box. The math is simple: take the area where the two boxes overlap (intersection), divide by the area of the union of the two boxes. The result is a number between 0 and 1. Zero means no overlap. One means the boxes match perfectly.

Common IoU thresholds tell you whether a prediction counts as correct. A prediction with IoU above 0.5 with the ground truth is generally counted as a correct detection in most benchmarks. A stricter threshold of 0.75 means the box has to be much more accurate. Reporting accuracy at multiple thresholds (0.5, 0.55, 0.6, ..., 0.95) and averaging gives a more complete picture of how well the detector localizes objects.



MATH MOMENT (optional) | IoU in plain English

Picture two rectangles on a piece of paper. One is the ground truth (where the object actually is). One is the predicted box (where the model said it was). The area where they overlap is the intersection. The area covered by either box is the union. IoU is the first divided by the second. If the boxes are identical, IoU is one. If they do not overlap at all, IoU is zero. The frameworks compute this for you; you just need to know what the number means when you see it in a benchmark.

Precision, Recall, and mAP

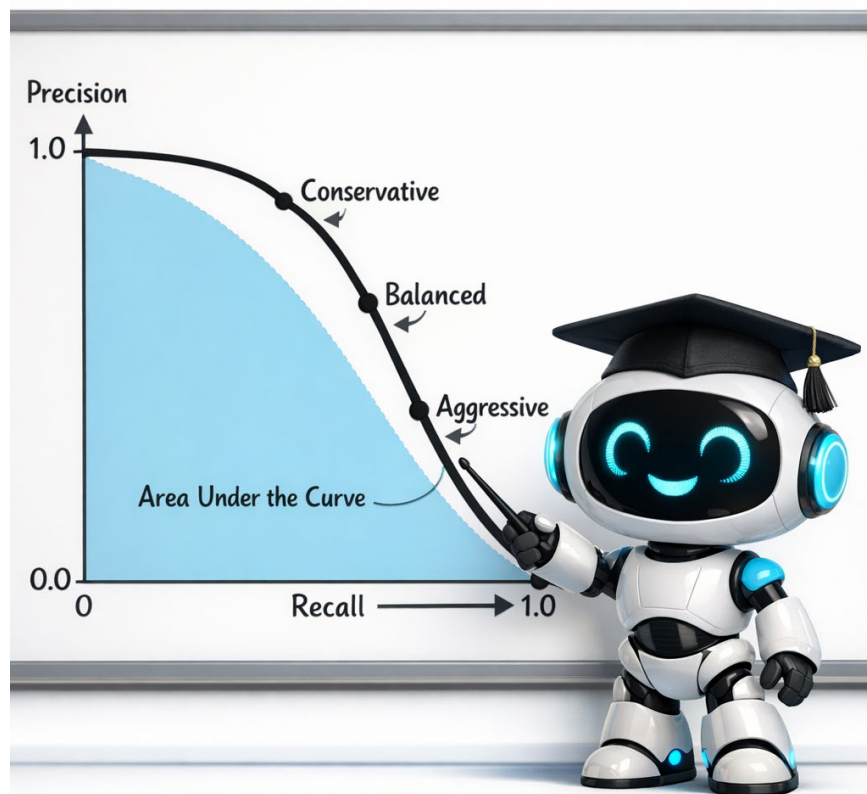
Three metrics together describe detector performance. Precision asks: of all the boxes the model drew, what fraction were correct (matched a real object with high enough IoU)? High precision means few false alarms. Recall asks: of all the real objects in the image, what fraction did the model find? High recall means few missed objects. These two metrics trade off against

each other. Lowering the confidence threshold catches more real objects (higher recall) but also creates more false alarms (lower precision).

Average precision (AP) combines precision and recall by computing the area under the precision recall curve as the confidence threshold sweeps. Mean average precision (mAP) averages the AP across all object classes (and often across multiple IoU thresholds). The mAP at IoU 0.5 (often written mAP@0.5) and the mAP averaged from IoU 0.5 to 0.95 (often written mAP@0.5:0.95) are the two numbers you will see most often in detection papers and leaderboards.

CAREER CONNECTION | **How to read a detection benchmark**

When a paper reports 'COCO mAP 52.3', that single number is doing a lot of work. It means: averaged across all 80 object classes in COCO, averaged across IoU thresholds from 0.5 to 0.95 in increments of 0.05, the precision recall area is 52.3 percent. Two detectors with the same mAP can have very different behavior on rare versus common classes, or on small versus large objects. In a job interview, knowing that mAP is an aggregate and not a single number per object is the difference between a beginner and an experienced practitioner.



The YOLO Family

YOLO stands for 'You Only Look Once'. It is the most popular one stage detector lineage. The name refers to the architecture's defining property: the entire detection happens in a single forward pass of the network, with no separate region proposal stage. YOLO was first published in 2016 by Joseph Redmon and has gone through many versions since.

The lineage in rough chronological order: YOLOv1 (2016, the original), YOLOv2 (2017, added anchor boxes), YOLOv3 (2018, used a deeper backbone), YOLOv4 and YOLOv5 (2020, by different teams, focused on practical training tricks), YOLOv6 and YOLOv7 (2022, further improvements), YOLOv8 (2023, by Ultralytics, very popular for ease of use), YOLO11 (2024, the current version at time of writing). The Ultralytics implementation has become the de facto standard for workforce CV, providing a clean Python API that makes training a custom YOLO detector a few lines of code.



INDUSTRY PRACTICE | The Ultralytics YOLO Python API

Training a custom YOLO detector with Ultralytics is genuinely simple: install the package, prepare your dataset in the documented format (a folder of images plus a text file per image listing object boxes and classes), call `model.train()` with your dataset path, and wait. This is what your lab uses. The training automation, the data augmentation defaults, the export to mobile and ONNX formats, all built in. This level of accessibility is why YOLO dominates workforce CV in 2026.

The DETR and RT-DETR Family

DETR (DEtection TRansformer), released by Meta in 2020, took a radically different approach: replace the entire hand crafted detection pipeline with a transformer. No anchor boxes, no NMS, no manual tuning. The transformer architecture (which you will learn about in Module 08) handles all of it internally with a small set of learnable queries that directly predict object locations. The output is a clean fixed length list of predictions.

The original DETR was conceptually elegant but slow at inference. RT-DETR (Real Time DETR), released by Baidu in 2023, fixed the speed problem. Modern RT-DETR matches or exceeds YOLO on both accuracy and speed in many benchmarks. The DETR family is gaining ground rapidly and represents the transformer wave reaching detection.

In 2026 you will see both anchor based YOLO style detectors and transformer based RT-DETR style detectors in production. Both work. Pick based on your team's familiarity, your deployment environment (YOLO has more mobile and edge support), and your accuracy requirements. As of this writing, YOLO is still slightly more common in production; RT-DETR is gaining.

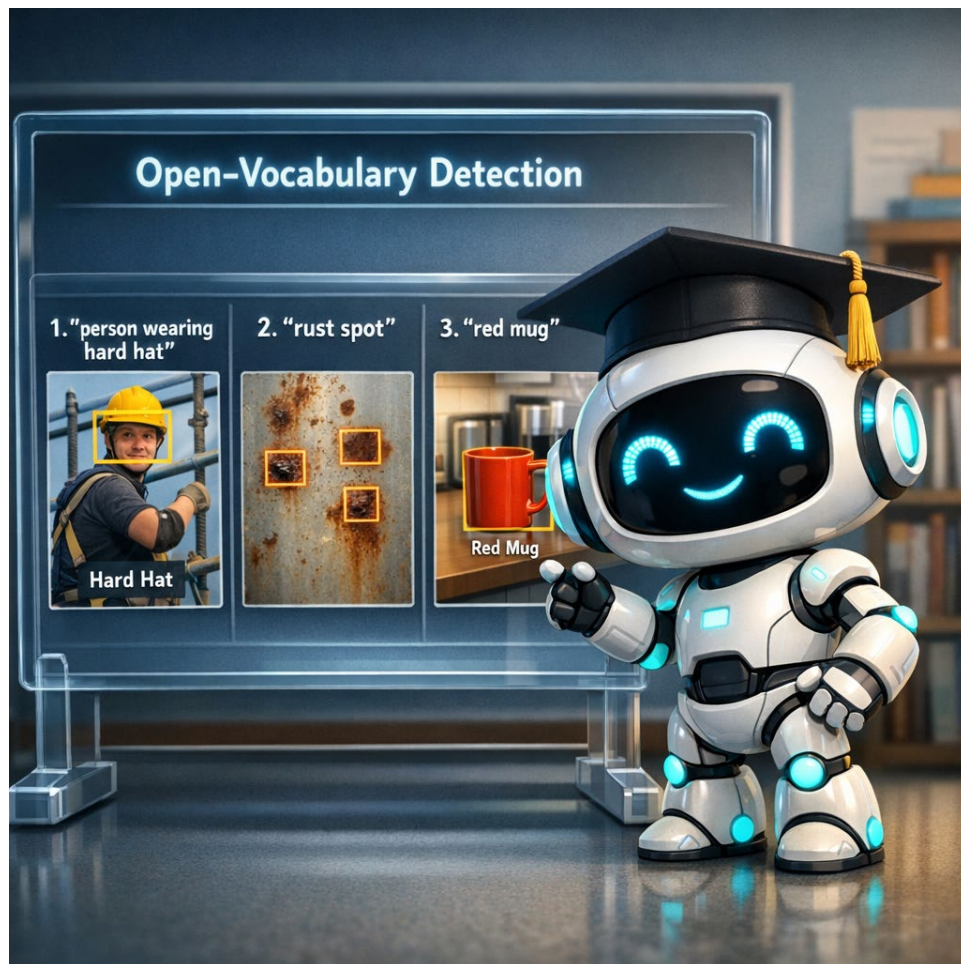


Open Vocabulary Detection: GroundingDINO

Classic detectors are stuck with the classes they were trained on. A YOLO model trained on COCO can detect 80 classes (person, car, dog, and so on). If you want to detect a 'red mug' or a 'cracked solar panel', you have to label data and train a new detector. This is expensive and slow.

Open vocabulary detection breaks the fixed class list. Models like GroundingDINO accept a text description as part of the input and find matching objects in the image, even classes the model was never trained on specifically. You can say 'a person wearing a hard hat' or 'small red defect on metal' and the model returns bounding boxes. This is enormous for industrial applications where new defect types appear all the time and labeling each one is impractical.

Open vocabulary detection is still less accurate than custom trained detectors on specific known classes. For high stakes production work, you still train a custom detector for the classes you know. But for prototyping, for new domains, for situations where labeled data is impossible, open vocabulary detection is transformative.



💡 THINK ABOUT IT | When to use open vocabulary versus trained detection

Faced with a new problem, here is a useful question. Will the set of object classes change frequently or be hard to enumerate? If yes, start with an open vocabulary detector. The flexibility is worth some accuracy loss. Are the classes well defined, stable, and high stakes (safety, regulation)? If yes, train a custom detector. The accuracy is worth the labeling effort. Sometimes both: use an open vocabulary detector for the long tail and a custom trained detector for the critical core classes.

Real Time Considerations

Real time detection is often misunderstood. The term implies fast, but real time really means end to end latency low enough for the use case. Detector inference speed is only part of the equation. Total latency includes camera capture (often 30 to 50 milliseconds for video), frame decoding (5 to 20 milliseconds), the detector itself (5 to 100 milliseconds depending on model), post processing (a few milliseconds), and downstream processing (variable). Add them all up and a detector running at 100 FPS on paper may deliver only 20 FPS end to end.

Tradeoffs that real time engineers tune. Input resolution: smaller images run faster but lose accuracy on small objects. Model size: nano models are blazing fast but less accurate. Batch size: small batches reduce latency but waste GPU utilization. Quantization: running in lower precision (INT8 instead of FP32) is much faster with small accuracy loss. The right balance depends on the deployment target (cloud GPU, edge device, mobile phone) and the application's latency requirements.



 MISCONCEPTION ALERT | **FPS is not the same as real time**

A detector that runs at 60 FPS on a benchmark might not be real time in your system. Measure end to end latency from frame capture to result delivery, not just model inference speed. Almost every production failure of real time CV is from someone reporting the detector benchmark FPS and assuming the system as a whole would match.

 KEY TAKEAWAY | **Part 03 in one paragraph**

Detection metrics start with IoU (overlap quality) and build to precision, recall, and mAP (the standard summary score). Three detector families dominate in 2026: YOLO for fast practical production work, DETR and RT-DETR for transformer based approaches, and GroundingDINO for open vocabulary tasks where the class list is fluid. Real time means end to end latency, not just FPS.

 KNOWLEDGE CHECK | **Quick check on metrics and families**

1. Define IoU in plain English. What does an IoU of 0.8 mean?
2. If a detector has high recall but low precision, what symptom would you see in its output?
3. Your team needs to detect arbitrary objects described in text prompts (such as 'orange road cone' or 'fallen tree'). Which detector family fits best?

Part 04 | Image Segmentation: Three Types and Why They Matter

Detection draws boxes around objects. Segmentation goes further: it labels every individual pixel in the image. The output is a mask the same size as the input, where each pixel has a class. This part introduces the three flavors of segmentation, the use cases that demand each, and the conceptual differences that will shape your model selection later in the module.

Semantic Segmentation

Semantic segmentation is the simplest form. It labels every pixel with a class but does not distinguish between different instances of the same class. If there are three cats in the image, all three are colored 'cat'. The output is a map where each pixel says what category it belongs to, but says nothing about which specific object it belongs to.

Semantic segmentation is appropriate when you care about the categories of regions but not about counting individual objects. Road segmentation for autonomous driving (you want to know which pixels are road, not how many roads exist). Land cover classification in satellite imagery. Background versus foreground separation for photo editing. Medical scans where the question is 'how much of this tissue is abnormal' rather than 'how many lesions are there'.

Instance Segmentation

Instance segmentation goes one step further. Every pixel is still labeled with a class, but pixels belonging to different individuals of the same class get different identifiers. Three cats become three separate masks, each with its own outline, even though all three have the class label 'cat'. Instance segmentation can be thought of as detection plus pixel level masks for each detected object.

Instance segmentation is the right choice when you need to count, track, or interact with individual objects. Counting cells in a microscope image. Tracking individual people through a crowd. Robotic grasping where you need the exact shape of the specific object you are about to pick up. Any application where 'one' versus 'many' matters at the pixel level.

Panoptic Segmentation

Panoptic segmentation combines both. Every pixel is labeled (like semantic), and every individual object instance is distinguished (like instance). The difference from instance segmentation alone is that panoptic also handles 'stuff' categories like sky, road, grass, that do not really have discrete instances. The result is a complete scene parse: every pixel knows both

its class and its instance, with consistent treatment of countable objects and uncountable regions.

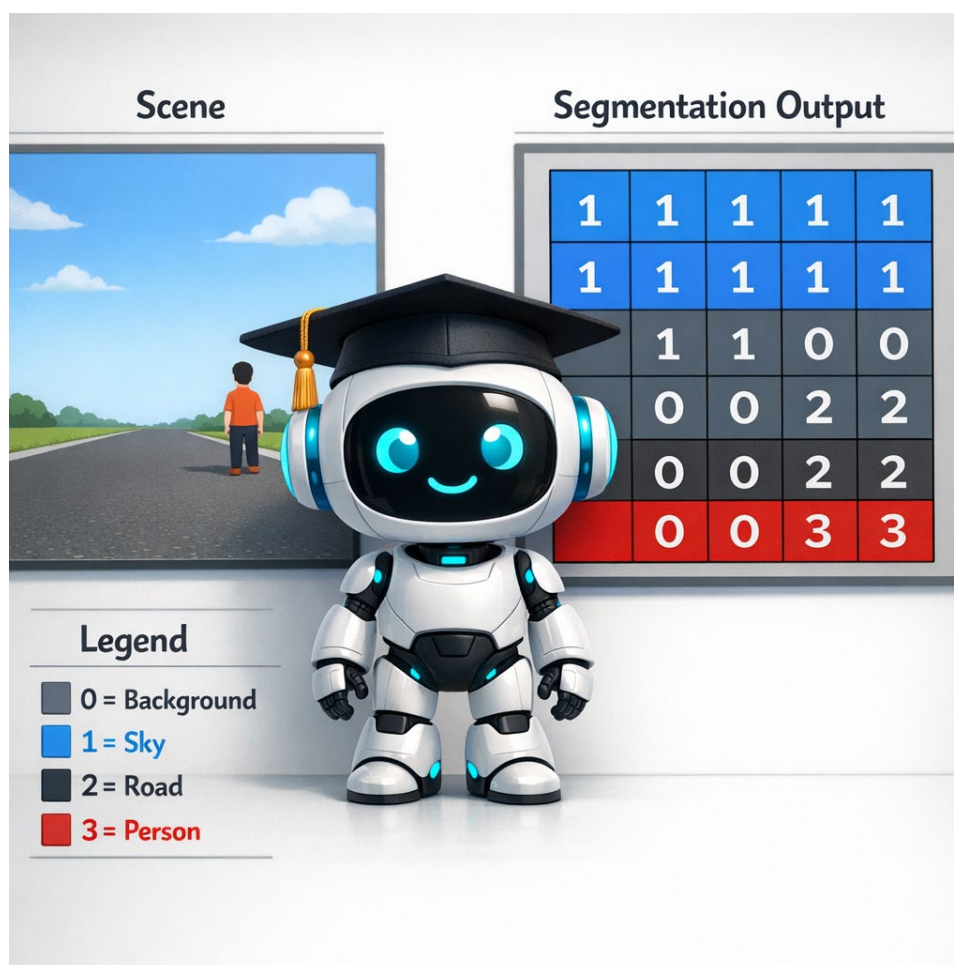
Panoptic segmentation is the richest representation of an image short of full 3D reconstruction. It is the standard for autonomous driving research, advanced robotics, and any application that needs a complete scene understanding. The cost is the most expensive form of annotation and the most complex models.



The Pixel Mask Output Format

In all three types of segmentation, the output is a mask the same size as the input image. For semantic segmentation, each pixel in the mask holds a class number (0 for background, 1 for road, 2 for sidewalk, and so on). For instance segmentation, each instance has its own mask, and the model outputs a collection of masks plus class labels. For panoptic, the output is a single mask with combined class and instance identifiers.

Working with masks in practice means handling arrays of class indices, often visualizing them by mapping each class to a distinct color, and computing metrics that compare predicted masks against ground truth masks. Frameworks like PyTorch and the segmentation libraries built on top of it (`torchvision.models.segmentation`, `segmentation_models_pytorch`, `MMSegmentation`) handle the heavy lifting; you focus on the data and the application.



INDUSTRY PRACTICE | Pixel masks are expensive to annotate

Labeling a single image with pixel level segmentation can take a skilled annotator 30 minutes to several hours, depending on complexity. Compare to a few seconds per image for classification labels and a minute or two for bounding boxes. This labeling cost is the single biggest practical reason detection often wins over segmentation in industry, even when masks would be better. Tools like Segment Anything (Part 06) are starting to change this; SAM can produce candidate masks that annotators just accept or refine, dramatically speeding labeling work.

Use Cases Across the Three Types

Choosing the right segmentation type comes down to the question your application is asking.

Medical imaging. Tumor boundary detection for surgery planning typically needs instance segmentation (each tumor as a separate region). Organ segmentation often only needs semantic (label each organ; only one of each). Treatment response measurement needs precise mask shapes regardless of type.

Autonomous driving. Lane and road segmentation is semantic. Detecting individual cars, pedestrians, and cyclists is instance. The full scene understanding (cars on road, sky above, trees to the side) is panoptic.

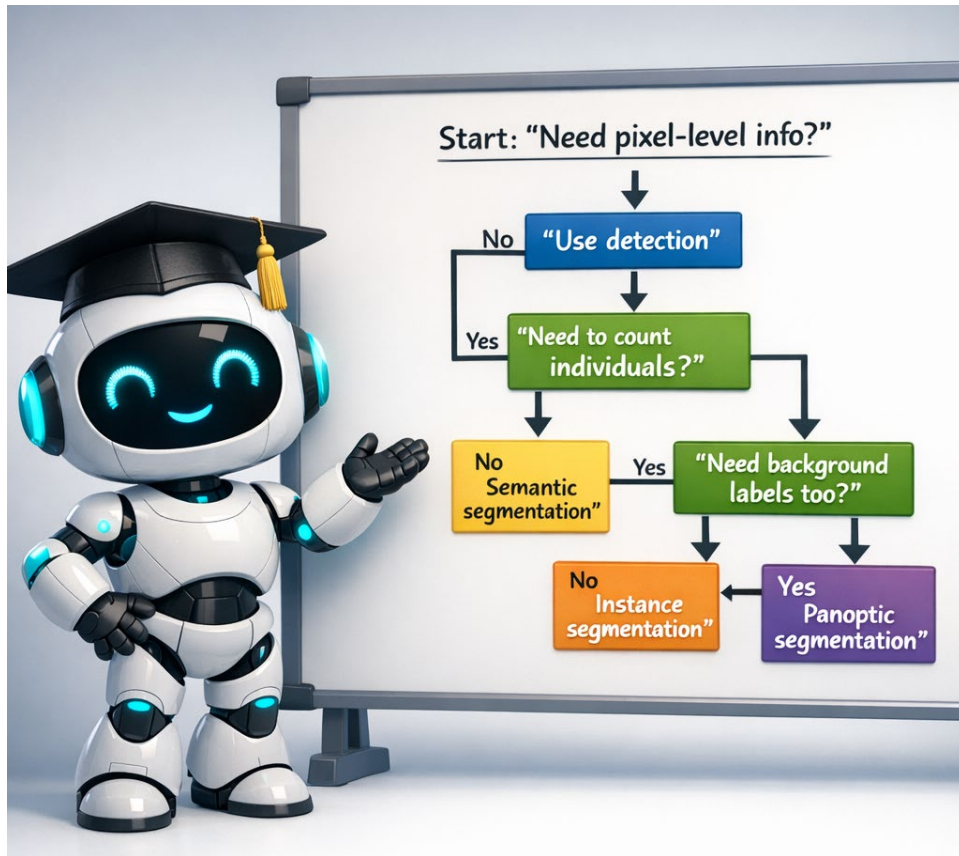
Photo editing. Background removal is semantic. 'Cut out this specific person from this photo' is instance. Apple's iPhone photo subject extraction is essentially instance segmentation.

Robotics. Picking up a specific object requires the exact shape of that one item, an instance segmentation task. Scene understanding for navigation often only needs semantic (where can the robot drive).



THINK ABOUT IT | Which type does your problem need

Ask three questions. First: do I need to know what each region is? If no, just use detection. Second: do I need to count or distinguish individual objects? If yes, instance or panoptic. If no, semantic. Third: do I need every pixel labeled including background regions like sky and ground? If yes, panoptic. If no, instance is enough. Most workforce applications can be solved with semantic or instance; panoptic is mainly research and advanced robotics today.



KEY TAKEAWAY | Part 04 in one paragraph

Three segmentation types live on a spectrum of richness. Semantic labels every pixel with a class. Instance distinguishes individual objects of the same class. Panoptic combines both and handles background regions consistently. Pick semantic when you care about regions, instance when you care about objects, panoptic when you need complete scene understanding. Mask annotation is expensive; use it deliberately.

KNOWLEDGE CHECK | Quick check on segmentation types

1. Three cats are in an image. How does the output differ between semantic and instance segmentation?
2. Which segmentation type would you use to count individual blood cells in a microscope image, and why?
3. Which segmentation type would you use to know 'how much of this image is sky'? Why is the cheaper option enough?

Part 05 | Segmentation Models: U-Net to Mask R-CNN

Now that you know what segmentation is, this part introduces the models that produce segmentation masks. Two models dominate the classical deep learning approach to segmentation: U-Net for semantic and medical work, and Mask R-CNN for instance segmentation. The part then covers the metrics used to measure segmentation models, and closes with other architectures worth knowing. Part 06 will then introduce the foundation model approach (Segment Anything) that changed the landscape.

U-Net: The Segmentation Workhorse

U-Net was invented in 2015 for biomedical image segmentation. It has remained the dominant architecture for semantic segmentation in domains with limited training data, and is still widely used in 2026. The architecture gets its name from its shape: an encoder path that shrinks the spatial dimensions while increasing the feature depth, a bottleneck at the bottom, and a decoder path that expands the spatial dimensions back to the original size. Drawn out, the network forms a U.

The Encoder, Bottleneck, and Decoder

The encoder is just a standard CNN. It applies convolution and pooling layers to progressively shrink the image while extracting deeper and more abstract features. By the bottleneck, the spatial resolution is small (say 16 by 16 from a 256 by 256 input), but each position holds a rich set of features about a large patch of the original image.

The decoder reverses the process. It uses transposed convolutions (also called up-sampling layers, operations that increase spatial size while learning patterns) to expand the feature maps back up to the original image size. At each step of the decoder, the feature maps grow larger and the channel depth shrinks.

The Key Trick: Skip Connections

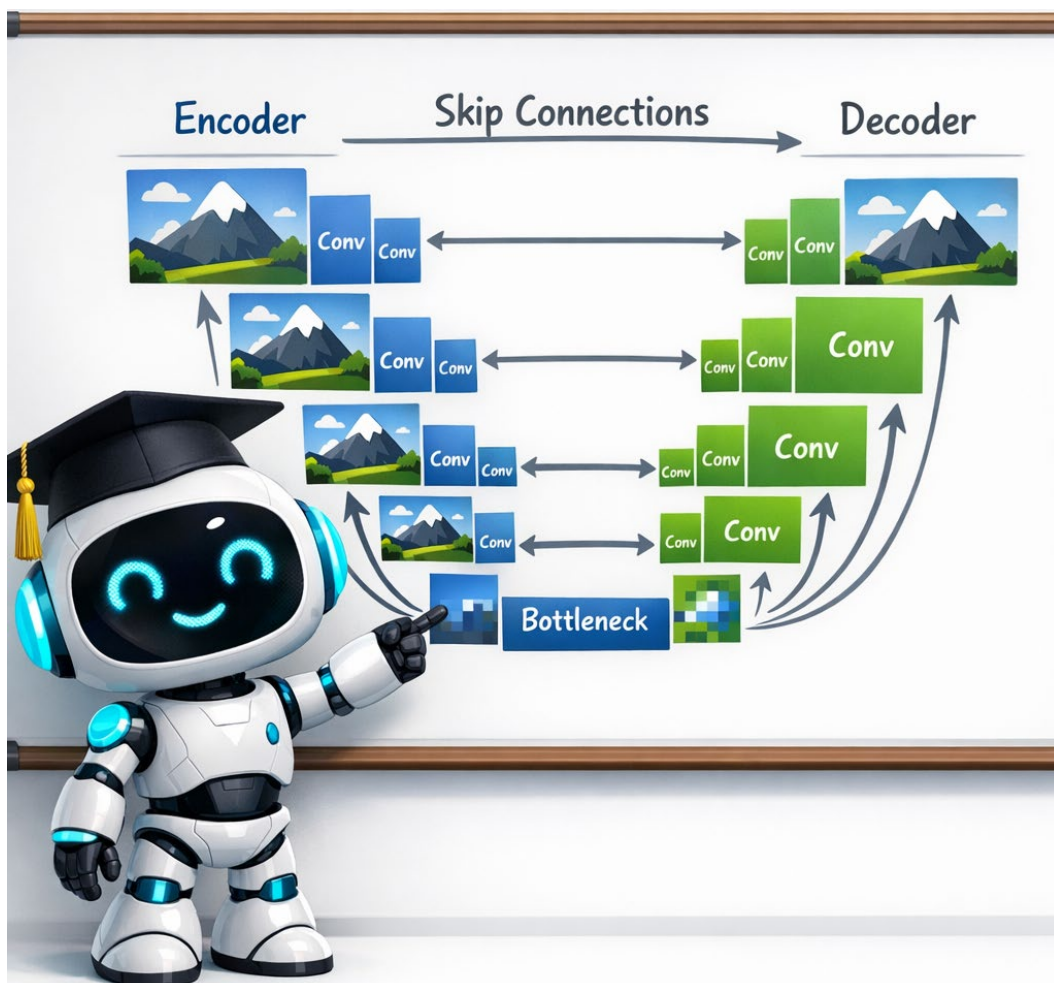
U-Net's defining innovation is the skip connections between encoder and decoder. At each level of the network, the encoder feature maps are passed directly across to the corresponding decoder level and concatenated with the decoder features. This preserves fine spatial details that would otherwise be lost during the downsampling and upsampling. The decoder gets both the abstract high level features from below and the detailed local features from across, letting it produce both correct labels and precise edges.

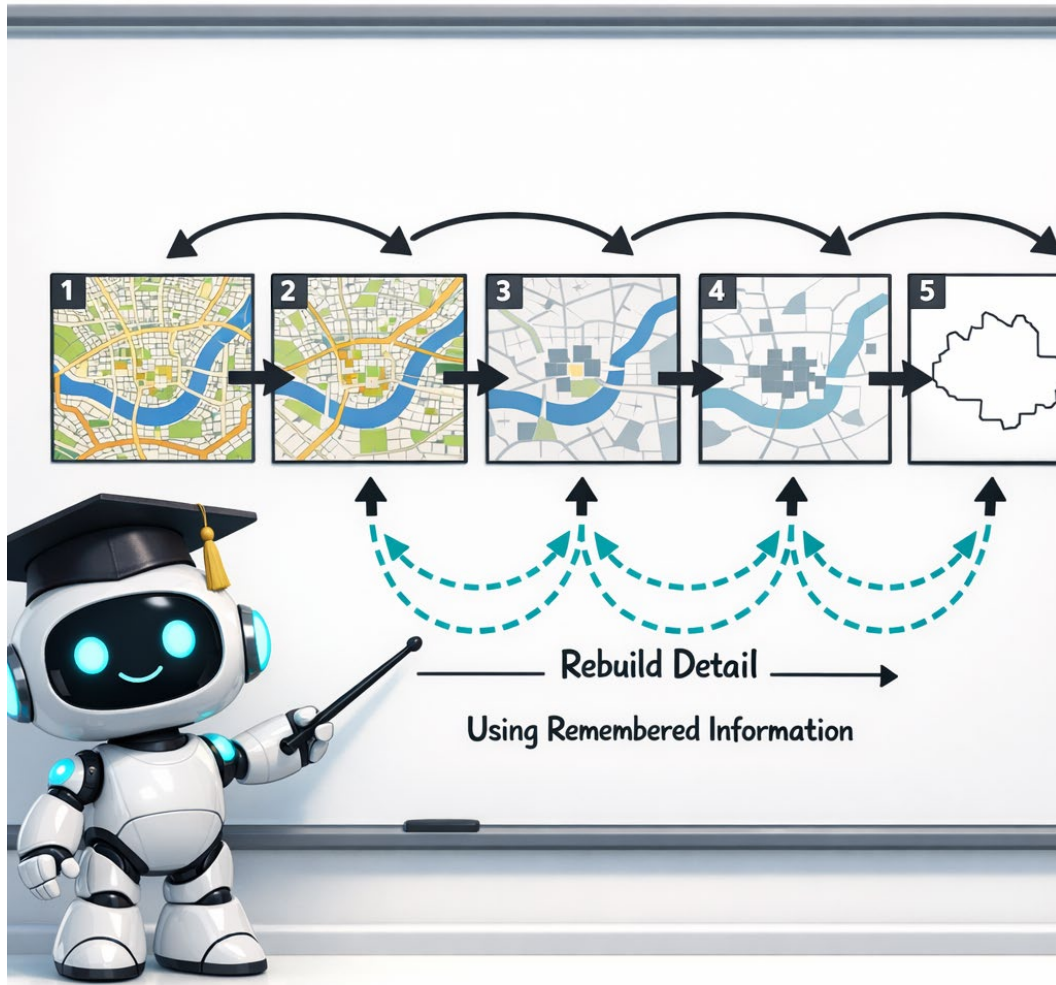
🔍 DEEPER LOOK | Skip connections in plain English

Picture zooming out on a map step by step. Each step, fine detail disappears. At maximum zoom out you can see the entire city but not individual buildings. You know what is there but not pixel level where each thing sits. That is what a regular CNN does as it goes deeper: it gets great at recognizing categories but loses the spatial precision needed to draw exact boundaries.

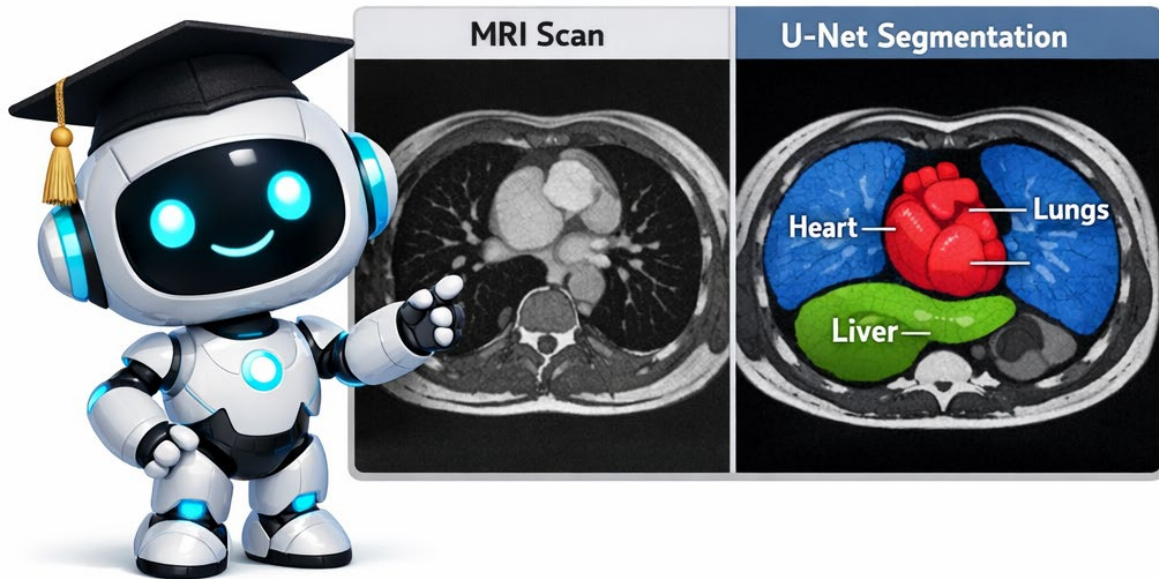
Now imagine that at every zoom out step, you save a snapshot of the map at that level. When you zoom back in to rebuild a detailed map, you consult those snapshots at each level. You can match the abstract knowledge from the deep view (this is the city's medical district) to the precise detail from the snapshots (this exact street, this exact building outline).

U-Net's skip connections are those snapshots. They are why a U-Net can produce pixel perfect outlines while a plain encoder decoder cannot. The decoder is not guessing the boundaries from abstract features; it is reading them directly from the encoder snapshots at every level of detail.





U-Net excels in scenarios with limited training data, which is common in medical imaging where labeled examples are expensive. The skip connections help the network generalize well even from small datasets. U-Net variants and improvements (Attention U-Net, nnU-Net, U-Net++) continue to be active areas of research and production deployment.



Mask R-CNN: Instance Segmentation

Mask R-CNN sits at the end of the R-CNN lineage from Part 02, adding a mask prediction head on top of Faster R-CNN's detection pipeline. The architecture has four main components.

The Backbone

A standard CNN (often ResNet50 or ResNet101) extracts features from the input image. This is the same backbone idea from Module 05, used as a feature extractor for downstream tasks.

The Region Proposal Network

A small subnetwork looks at the backbone features and proposes candidate regions that might contain objects. This is the same idea as the region proposal in Faster R-CNN, the detector that Mask R-CNN is built on.

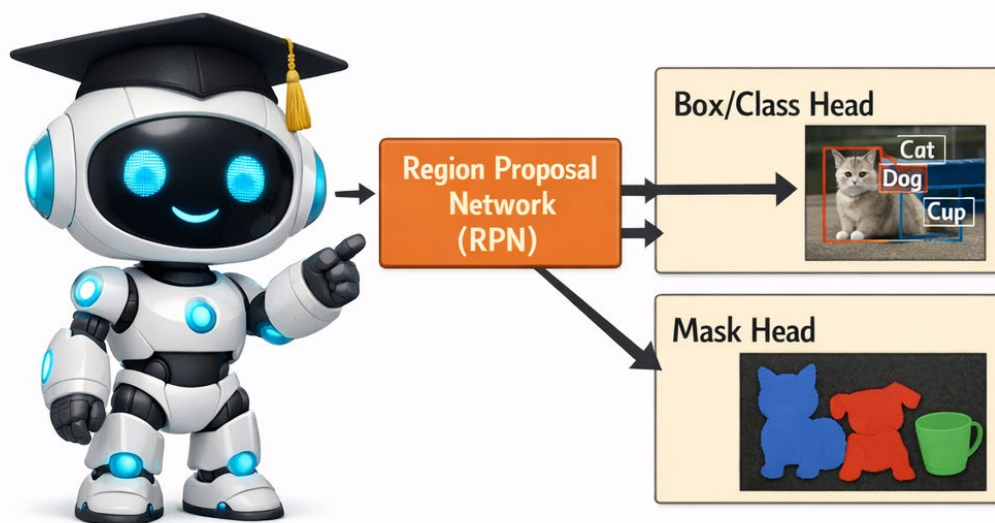
The ROI Classifier

For each proposed region, a classifier head decides what the object is and refines the bounding box. This is the standard detection head, producing a class label and a refined box.

The Mask Head

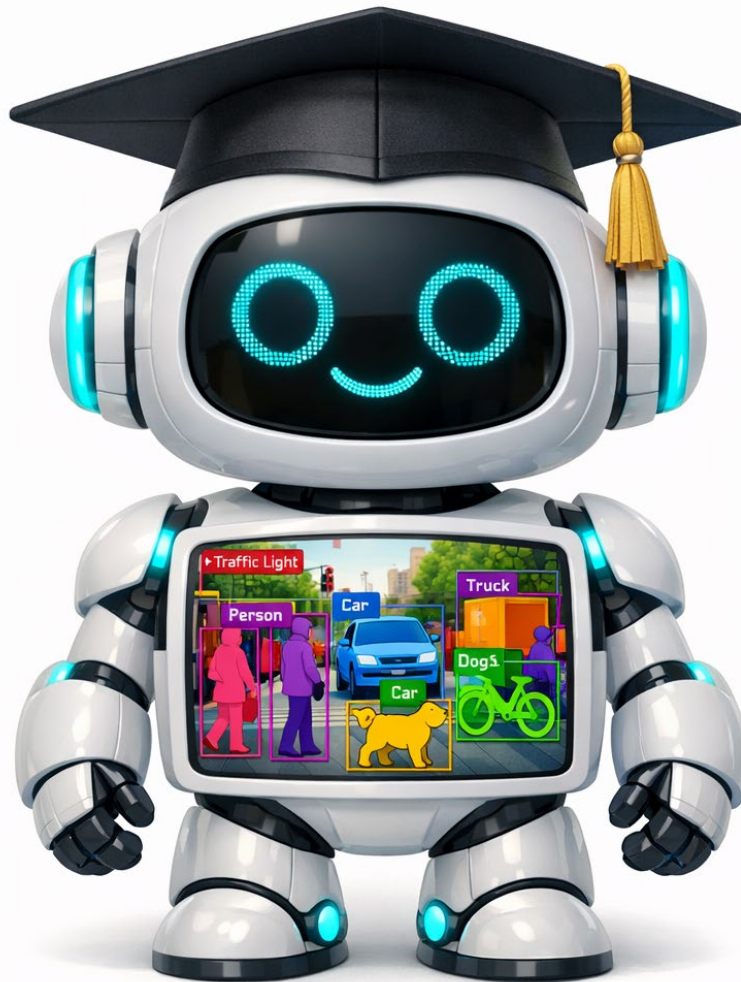
The innovation of Mask R-CNN is the addition of a fourth component: a mask head that, for each detected object, produces a pixel level mask just for that object. The mask is computed in the coordinate frame of the bounding box and then transferred back to the image.

Mask R-CNN Architecture



DEEPER LOOK | Why Mask R-CNN feels both like detection and segmentation

Mask R-CNN is literally a detector with a mask added. The first three components (backbone, region proposal, ROI classifier) are the same as Faster R-CNN, a pure detector. Only the fourth component (mask head) adds segmentation. This makes Mask R-CNN a strong choice when you already need detection and want to add mask outputs without switching architectures. You get boxes and masks together from the same model. No equations to memorize; the framework handles the math. What matters is understanding that the same network produces both kinds of output from the same features.



When U-Net Versus Mask R-CNN

Two clear guidelines help you choose. Use U-Net when you have a small dataset and need semantic segmentation in a specialized domain (medical imaging is the canonical example). The architecture is simpler, easier to train with limited data, and produces clean pixel masks.

Use Mask R-CNN when you need instance segmentation across many object classes, especially if you also need detection. The architecture is heavier and needs more data, but it produces both boxes and masks from a single model. It handles complex scenes with many objects naturally.

Both architectures are mature, with strong implementations in PyTorch and TensorFlow. Both can be initialized from pretrained weights (often a ResNet backbone pretrained on ImageNet) and fine tuned with transfer learning from Module 05. The fine tuning workflow is essentially

identical to what you learned for classification: load the model, optionally swap the head for your task, train on your data with a low learning rate.

INDUSTRY PRACTICE | Where the segmentation libraries live

In PyTorch, `segmentation_models_pytorch` (smp) is a popular library that wraps U-Net and many variants with pretrained encoders. For instance segmentation, `torchvision.models.detection` includes Mask R-CNN with pretrained weights. For more advanced research work, OpenMMLab provides MMSegmentation (semantic) and MMDetection (instance and panoptic). Most workforce jobs use smp for semantic tasks and torchvision Mask R-CNN for instance tasks. Knowing both libraries is the workforce baseline.

Measuring Segmentation Performance

Detection has IoU, precision, recall, and mAP. Segmentation has its own family of metrics, parallel in spirit but adapted for pixel masks instead of bounding boxes. Four metrics dominate workforce practice in 2026.

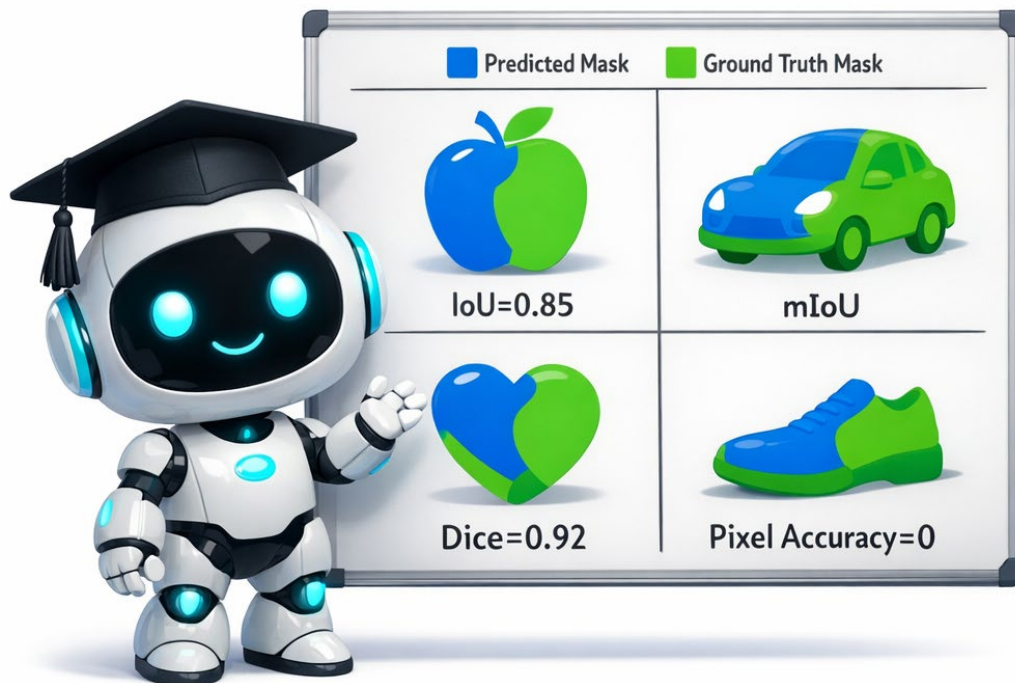
IoU for masks. The intersection over union idea from Part 03 carries directly into segmentation, but applied to pixel masks instead of boxes. Count the pixels the predicted mask and the ground truth mask share (the intersection), and divide by the total pixels covered by either (the union). The result is between 0 and 1. One means the masks line up perfectly. Zero means no overlap. This is the foundation metric for segmentation.

mIoU (mean IoU). A real world dataset has many classes. You compute IoU for each class separately, then average across all classes. That average is the mean IoU, often written mIoU. This is the single number you will see most often in segmentation papers and benchmarks. The Cityscapes, ADE20K, and Pascal VOC leaderboards all report mIoU as the headline number.

Dice score. Closely related to IoU but calculated slightly differently: twice the intersection divided by the sum of the two areas. Like IoU it ranges from 0 to 1, higher is better. Dice is the default metric in medical imaging because it is slightly more forgiving than IoU when the structures being segmented are small (a small tumor with a slightly off mask penalizes less under Dice than under IoU). When you see medical imaging papers reporting Dice score, this is why.

Pixel accuracy. The fraction of pixels labeled correctly across the whole image. Easy to compute and easy to understand, which is why beginners reach for it. Unfortunately it is misleading whenever one class dominates. A photo where 95 percent of the pixels are sky can show 95 percent pixel accuracy with a model that just predicts sky for everything. For balanced

datasets pixel accuracy is fine as a quick sanity check, but mIoU is the metric that actually measures quality.



CAREER CONNECTION | Which metric to report

mIoU is the default for general purpose segmentation work and for any academic comparison. Dice is the default for medical imaging. Pixel accuracy is useful as a quick check or alongside mIoU, never alone. In a job interview, knowing the difference between these three and being able to say 'I would report mIoU on a general dataset like Cityscapes and Dice on the medical imaging dataset' demonstrates a fluency that beginners rarely show. The lab in this module reports mIoU; expect to see Dice if you take on a medical imaging project in your career.

Other Architectures Worth Knowing

Two more families round out the classical deep segmentation landscape. Fully Convolutional Networks (FCN) preceded U-Net and showed that any classification CNN could be converted to produce dense pixel predictions by replacing fully connected layers with convolutions. DeepLab

models use atrous (dilated) convolutions to enlarge the receptive field without losing resolution, and are strong for varied object sizes. Both appear in research papers and production systems, especially in domains with specific quality requirements.

In 2026, the classical approaches (U-Net, Mask R-CNN, DeepLab variants) coexist with the foundation model approach (Part 06). For semantic segmentation in a specialized domain with limited data, U-Net is still often the right answer. For instance segmentation with detection, Mask R-CNN holds. For zero shot or interactive segmentation, Segment Anything wins. Knowing which to reach for is workforce judgment.

KEY TAKEAWAY | **Part 05 in one paragraph**

U-Net is the workhorse for semantic segmentation, especially with limited data and in medical imaging. Its U shape (encoder, bottleneck, decoder) with skip connections preserves fine details. Mask R-CNN sits at the end of the R-CNN lineage, extending detection with a mask head and producing both boxes and pixel masks for instance segmentation. Segmentation has its own metrics family: mIoU is the default, Dice for medical, pixel accuracy as a quick check only. FCN and DeepLab round out the classical landscape. Use transfer learning from Module 05 to fine tune any of these on your task.

KNOWLEDGE CHECK | **Quick check on segmentation models**

1. Describe the three main components of U-Net's architecture (encoder, bottleneck, decoder) and what each does.
2. What architectural innovation defines U-Net and what problem does it solve?
3. A team has 500 labeled medical scans and needs to segment a single organ in each. U-Net or Mask R-CNN, and why?
4. Which segmentation metric would you report on a Cityscapes benchmark, and which on a medical imaging benchmark? Why are they different?

Part 06 | Foundation Models and Choosing the Right Approach

This part covers the model that changed segmentation in 2023, the Segment Anything Model (SAM) from Meta, and its 2024 successor SAM 2 that added video. Then it pulls all of Module 06 together with a decision rubric for choosing the right detection and segmentation approach for any problem, plus common pitfalls to avoid.

Segment Anything: The Foundation Model Moment

Until 2023, segmentation was always a task you trained a model for. New domain, new model, new training run. Meta's Segment Anything Model (SAM), released in April 2023, changed that. SAM is a foundation model for segmentation: a single model that was trained on 11 million images and over one billion segmentation masks, and that can segment any object in any image without being retrained for it. This is called zero shot segmentation.

Three Ways to Use SAM

SAM does not produce a single fixed mask output. It is interactive, designed to be guided by prompts. Three prompt types are supported.

Point prompt. You click on an object in the image. SAM returns the mask for whatever object contains that point. Click on a cat and you get the cat's mask. Click on a chair and you get the chair's mask. Multiple points (some positive, some negative) can refine the result.

Box prompt. You draw a rough bounding box around an object. SAM returns the precise mask for the object inside the box. This is especially useful when combined with a detector: the detector finds objects with bounding boxes, then SAM converts those boxes into pixel level masks.

Everything mode. You give no prompt. SAM automatically segments every object it can find in the image and returns a collection of masks. This is the right mode for unconstrained scene parsing or for accelerating data annotation.

Comparing SAM Prompt Types



Point prompt



Box prompt



Text prompt





Why SAM Matters for the Workforce

SAM changed three things about practical segmentation work. First, it eliminated the need for custom training in many cases. For prototyping, for one off projects, for situations where the class list is fluid, SAM works out of the box.

Second, it transformed data annotation. Labeling pixel masks used to be the most expensive form of annotation, taking minutes to hours per image. With SAM, annotators click on objects, SAM produces candidate masks, and humans only accept or refine. Annotation throughput jumped by an order of magnitude in many shops in 2023 and 2024.

Third, it became a callable service that you compose with other models. Run a YOLO detector to find bounding boxes, then pass each box to SAM to get a precise mask. Run an open vocabulary detector like GroundingDINO to find arbitrary text described objects, then SAM to

get their masks. The composition of foundation models is itself a foundation pattern in modern CV.



INDUSTRY PRACTICE | How SAM is actually used in production

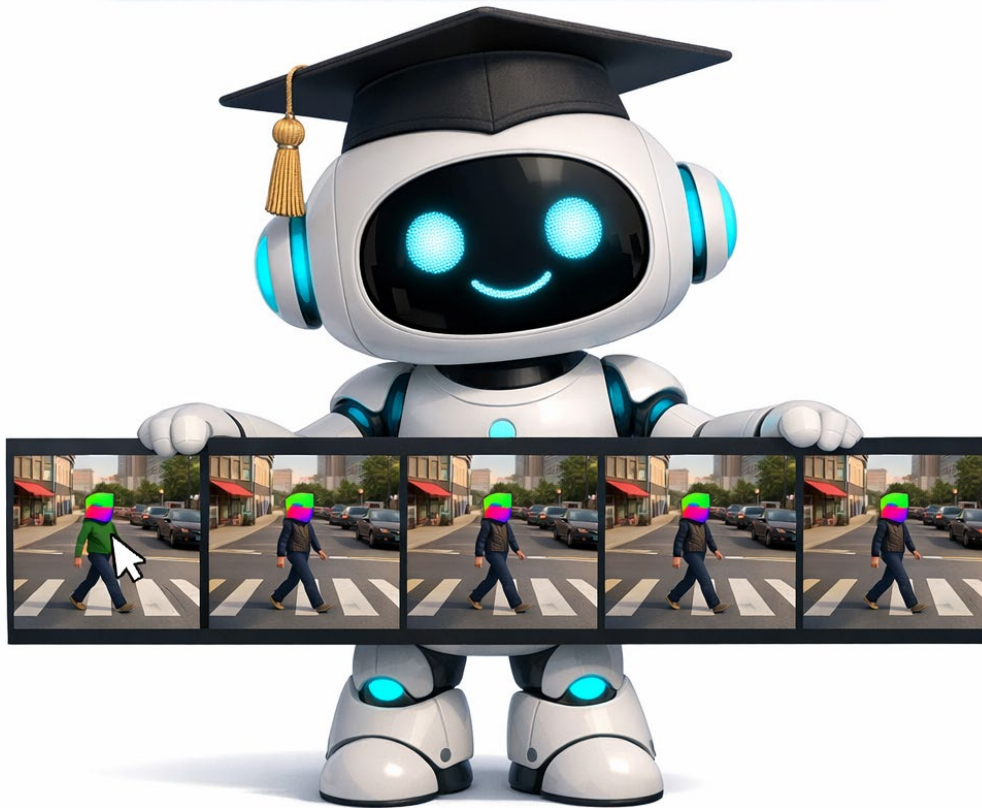
Three patterns dominate. Pattern 1: data annotation pipeline. Annotators use SAM to speed up manual labeling for custom model training. Pattern 2: interactive applications. Photo editors, medical viewers, and similar tools embed SAM for user driven mask creation. Pattern 3: composition with detectors. A YOLO or RT-DETR detector locates objects, SAM produces precise masks for downstream measurement or visualization. Pure 'SAM only' production pipelines are still rare; SAM as a component in a larger pipeline is now standard.

SAM 2: Adding Video

Meta released SAM 2 in mid 2024. The major addition was video. SAM 2 keeps all the image capabilities of the original SAM and adds the ability to track masks across video frames. Click on an object in frame one of a video, and SAM 2 produces the mask for that object in every subsequent frame, handling occlusion, motion, and changes in appearance.

This was transformative for video annotation, which used to be a brutal frame by frame process. With SAM 2, annotating an object across an entire video clip is often a single click on the first frame. Video segmentation models trained on SAM 2 outputs have become a significant production technique. Module 10 (Video Analysis and Object Tracking) revisits SAM 2 in detail for tracking applications.

SAM 2: one click, **track through video**

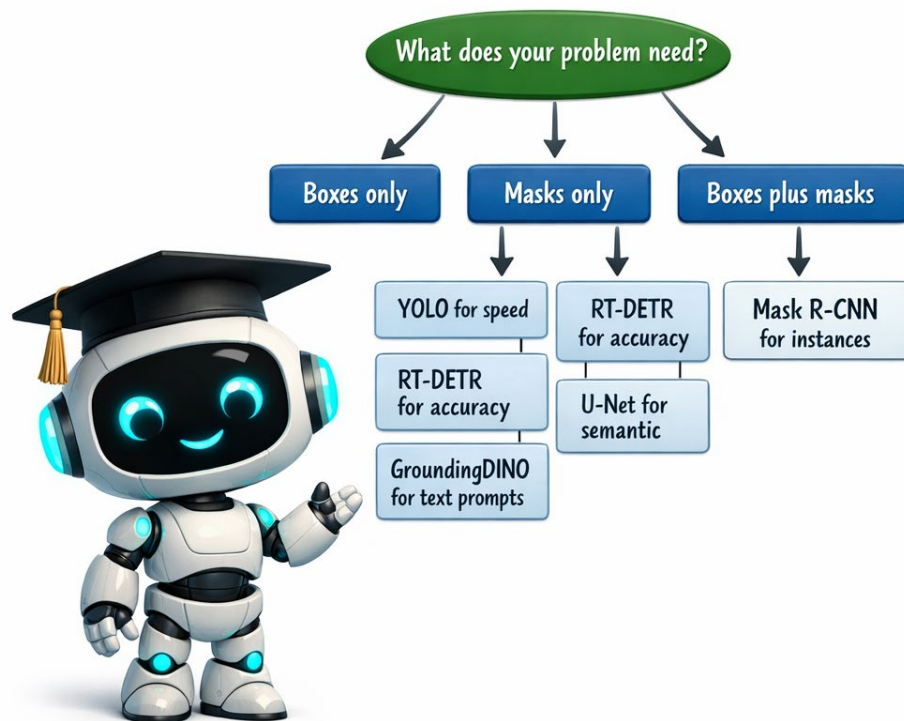


CAREER CONNECTION | SAM and SAM 2 vocabulary for interviews

Knowing the names SAM and SAM 2, what they do, who built them (Meta AI), and the prompt types (point, box, everything mode) is workforce baseline as of 2026. The deeper understanding (the masked autoencoder pretraining, the prompt encoder architecture) is not expected at the intro level. But being able to say 'SAM 2 is the foundation model for image and video segmentation; it accepts point or box prompts and returns precise masks zero shot; it is composed with detectors in modern CV pipelines' is the kind of fluent answer that recruiters listen for.

The Decision Rubric: Choosing Detection and Segmentation

Pulling everything in Module 06 together, here is the decision tree you should be able to walk in an interview.



Do You Need Boxes, Masks, or Both?

If boxes alone answer your question, you only need detection. Choose YOLO11 (Ultralytics) for fast simple production, RT-DETR for transformer based with similar speed, or GroundingDINO for open vocabulary needs.

If you need pixel masks, you need segmentation. Choose U-Net for semantic segmentation in a specialized domain with limited data. Choose Mask R-CNN for instance segmentation when you also need detection. Choose SAM or SAM 2 when you need zero shot masks across diverse objects or when you are doing interactive applications.

If you need both boxes and masks (common in industrial inspection, robotics, advanced analytics), you have two paths. Use Mask R-CNN if you want both from one model trained on your data. Or compose: use a detector (YOLO, RT-DETR) to find boxes, then SAM to convert

each box to a mask. The composition pattern is often more flexible and gets you better results in 2026.

How Much Data Do You Have?

Small dataset (under a few hundred labeled images per class): try SAM with prompts for segmentation, or feature extraction transfer learning for detection. Avoid training Mask R-CNN from scratch.

Medium dataset (hundreds to thousands per class): fine tune a pretrained YOLO or Mask R-CNN with transfer learning from Module 05. Use heavy data augmentation.

Large dataset (tens of thousands or more per class): fine tune more aggressively, or train from a strong pretrained baseline. Consider open vocabulary detectors if your class set is large or evolving.

Common Pitfalls in Detection and Segmentation

Three pitfalls trip up beginners specifically in detection and segmentation work.

Pitfall 1: Bad Annotations

Detection and segmentation are sensitive to annotation quality. A box that includes too much background or excludes part of the object teaches the model wrong lessons. A pixel mask that misses fine edges trains the model to miss those edges too. Spend time on annotation guidelines, do quality control passes, and prefer fewer high quality annotations over many sloppy ones.

Pitfall 2: Mismatched Confidence Thresholds

Detectors output many candidate boxes; you filter by confidence threshold. A threshold that is too high misses real objects (low recall). A threshold that is too low produces false alarms (low precision). Tune the threshold for your application, and remember that mAP combines performance across all thresholds, but your production system uses just one. Pick the threshold for your business need, not for the benchmark.

Pitfall 3: Real Time Promises Without Real Time Measurement

Detectors quote inference speed in FPS on a specific benchmark hardware. Your actual deployment will be different: different hardware, different image sizes, different batch sizes, different surrounding software stack. Always measure end to end latency in your real deployment, with realistic input, before promising any real time behavior. The gap between benchmark FPS and real world FPS is often a factor of two or more.

 MISCONCEPTION ALERT | **Foundation models will not solve every problem**

It is tempting in 2026 to think 'just use SAM' or 'just use GroundingDINO' for everything. They are general but not always best. A custom trained YOLO on a clean dataset of factory defects will outperform an open vocabulary detector on those same defects, every time. A specialized U-Net on medical scans will outperform SAM on those exact organ shapes. Foundation models are wonderful for prototyping and for tasks where training data is scarce. For mature high stakes production, custom trained models still rule.

 KEY TAKEAWAY | **Part 06 in one paragraph**

Segment Anything (SAM, SAM 2) is the foundation model that changed segmentation in 2023 and 2024. It accepts point, box, or no prompts and returns zero shot masks. It composes naturally with detectors: detect with YOLO or RT-DETR, segment with SAM. The decision rubric: boxes alone means detection (YOLO, RT-DETR, GroundingDINO); masks needed means segmentation (U-Net, Mask R-CNN, or SAM); both means composition. Match the approach to your data, your task, and your hardware. Avoid the three pitfalls: bad annotations, mismatched thresholds, FPS without end to end measurement.

 KNOWLEDGE CHECK | **Quick check on foundation models and choosing**

1. Name the three prompt types SAM accepts and describe what each produces.
2. A team needs to add precise pixel masks to a YOLO based defect detection pipeline without retraining. What is the simplest path forward?
3. Name the three common pitfalls discussed in this part and give a one sentence fix for each.

Review Questions

Work through these on your own first. Then check your answers against the booklet and your notes. If any question feels uncertain, that is the section to reread.

1. Describe the three CV tasks (classification, detection, segmentation) and what kind of output each one produces.
2. Why do all three tasks typically use the same CNN backbone? What changes between them?
3. What are the two common conventions for representing a bounding box, and what does each store?
4. Explain what anchor boxes are and why detectors use them.
5. Walk through what non maximum suppression does, in plain English.
6. Define intersection over union (IoU) and describe what an IoU of 0.7 means.
7. Explain the difference between precision and recall in detection. Why do they trade off against each other?
8. Name the four versions in the R-CNN lineage and describe what each one fixed.
9. Name three modern detector families and describe the strengths of each.
10. What is open vocabulary detection and when would you use it?
11. What does 'real time' really mean for a detection system, and why is FPS alone not enough?
12. Compare semantic, instance, and panoptic segmentation. Give one use case where each is the right choice.
13. Describe the U-Net architecture and explain what role skip connections play. Use the map analogy in your own words.
14. How does Mask R-CNN extend a detection model to produce segmentation masks?
15. Name the four segmentation metrics covered in Part 05 and explain which to report on a general dataset versus a medical imaging dataset.
16. What does Segment Anything (SAM) do that classical segmentation models do not? Name the three prompt types it accepts.
17. A factory inspection system needs both bounding boxes and precise pixel masks for defects. Describe two valid approaches and the tradeoffs.

Glossary

Anchor box

A preset bounding box template placed at every position on a grid across the image. Detectors use anchors as starting candidates that the model then either accepts (with adjustments) or rejects. Modern transformer detectors replace anchors with learnable queries.

Backbone

The convolutional neural network at the bottom of a detection or segmentation model that extracts visual features from the input image. Often a ResNet or similar pretrained classification network used as a feature extractor. The same backbone idea appears in YOLO, DETR, U-Net, Mask R-CNN, and most modern CV models.

Bottleneck

The narrow middle of an encoder decoder architecture (like U-Net) where the spatial resolution is smallest and the feature representation is most abstract. The bottleneck holds the model's high level understanding of what is in the image before the decoder begins reconstructing pixel level output.

Bounding box

A rectangle described by four numbers that locates an object in an image. Two common conventions: `xyxy` (top left x, top left y, bottom right x, bottom right y) and `xywh` (top left x, top left y, width, height).

Classification

The CV task of assigning a single class label to an entire image. The cheapest task in terms of annotation cost and model complexity.

Confidence score

A number between zero and one that the detector emits with each prediction. Higher confidence means the model is more certain about the prediction. Used to filter candidate boxes and to compute precision recall curves.

Decoder

The expanding half of an encoder decoder architecture. Takes the abstract feature representation from the bottleneck and progressively builds it back up to the spatial size needed for the output, such as a segmentation mask.

Detection

The CV task of finding all objects of interest in an image and producing a bounding box and class label for each. Output is a variable length list of labeled boxes.

DETR (DEtection TRansformer)

A 2020 detector from Meta that replaced anchor boxes and NMS with a transformer architecture. Conceptually clean and accurate but slow at inference. Improved by RT-DETR.

Dice score

A segmentation metric calculated as twice the intersection divided by the sum of both areas (predicted and ground truth). Closely related to IoU, ranges from 0 to 1, higher is better. The default metric in medical imaging.

Encoder

The shrinking half of an encoder decoder architecture. Takes the input image and progressively reduces its spatial size while extracting higher level features. In U-Net the encoder is a standard CNN.

Fast R-CNN

A 2015 detector that improved on R-CNN by running the CNN once over the whole image and looking up region features from the resulting feature map. Much faster than R-CNN. Was itself superseded by Faster R-CNN later the same year.

Faster R-CNN

A 2015 detector that replaced the slow classical region proposal step in Fast R-CNN with a learned subnetwork called the Region Proposal Network (RPN). End to end deep learning. The architecture Mask R-CNN is built on top of.

FCN (Fully Convolutional Network)

A 2014 architecture that converted classification CNNs into segmentation models by replacing fully connected layers with convolutions. Predecessor to U-Net.

Foundation model

A large pretrained model intended to be used across many tasks and domains without retraining. SAM, GroundingDINO, CLIP, and the large language models are all foundation models. The defining feature is generality: train once on huge data, use across many downstream applications.

GroundingDINO

An open vocabulary detector that accepts a text description as input and finds matching objects in the image, including classes the model was never specifically trained on. Useful for prototyping and new domains.

Instance segmentation

Segmentation that labels every pixel and distinguishes between individual instances of the same class. Three cats become three separate masks.

IoU (Intersection over Union)

A metric that measures how well a predicted box or mask matches a ground truth box or mask. Computed as the overlap area divided by the union area. Ranges from 0 (no overlap) to 1 (perfect match).

mAP (mean Average Precision)

The standard summary metric for object detection. Computed by averaging precision recall curves across all object classes and across multiple IoU thresholds. Higher is better, scale 0 to 1 (often shown as a percentage). The headline number on detection benchmarks like COCO.

Mask

The output of a segmentation model. An array the same size as the input image where each pixel holds a class index (semantic) or instance identifier (instance and panoptic).

Mask R-CNN

A 2017 instance segmentation model that extends Faster R-CNN with a mask prediction head. Produces bounding boxes, class labels, and pixel masks for each detected object. The end of the R-CNN lineage.

mIoU (mean IoU)

The standard summary metric for semantic segmentation. Computes IoU for each class separately, then averages across all classes. The single number most segmentation benchmarks (Cityscapes, ADE20K, Pascal VOC) report.

Non Maximum Suppression (NMS)

A cleanup step that removes duplicate detector predictions. Sorts predictions by confidence and discards lower confidence boxes that overlap too much with higher confidence ones.

One stage detector

A detector that produces all boxes and labels in a single forward pass of the network. Faster than two stage detectors and dominant in production. YOLO is the canonical example.

Open vocabulary detection

Detection that accepts a text prompt and finds matching objects, including classes the model was never trained on. GroundingDINO is the canonical example.

Panoptic segmentation

Segmentation that combines semantic and instance. Every pixel is labeled with both a class and an instance identifier. Background regions like sky and road are handled consistently.

Pixel accuracy

The fraction of pixels labeled correctly across an image. Simple to compute but misleading when one class dominates the image. Use as a quick check, not as the primary segmentation metric.

Precision

Of all the boxes the detector drew, what fraction were correct. High precision means few false alarms.

R-CNN

The 2014 detector that started the two stage lineage. Uses a classical region proposal algorithm followed by a CNN classifier on each region. Slow but established that deep learning could win detection benchmarks. The foundation for Fast R-CNN, Faster R-CNN, and Mask R-CNN.

Recall

Of all the real objects in the image, what fraction did the detector find. High recall means few missed objects. Trades off against precision.

Region Proposal Network (RPN)

A small subnetwork in Faster R-CNN and Mask R-CNN that proposes candidate object regions from CNN features. Replaced the slow classical region proposal step of earlier detectors.

RT-DETR (Real Time DETR)

A 2023 detector from Baidu that fixed the speed problem of DETR. Matches or exceeds YOLO on accuracy and speed in many benchmarks. The transformer detector you would actually pick for production today.

SAM (Segment Anything Model)

Meta's 2023 foundation model for segmentation. Accepts point, box, or no prompts and returns zero shot masks. Trained on 11 million images and over one billion masks.

SAM 2

Meta's 2024 update to SAM that adds video. Click on an object in one frame and SAM 2 tracks the mask across the rest of the video, handling occlusion and motion.

Semantic segmentation

Segmentation that labels every pixel with a class but does not distinguish between individual instances. Three cats are all colored the same 'cat' color.

Skip connection

A direct pathway in a neural network that bypasses one or more intermediate layers. In U-Net, skip connections carry high resolution features from the encoder directly across to the matching decoder level, preserving fine spatial details that would otherwise be lost.

Transfer learning

Reusing a model trained on one task as the starting point for training on a different task. In CV, almost every detector and segmenter starts from a backbone pretrained on ImageNet classification, then fine tunes on the target task. Covered in Module 05; the most important practical technique in workforce CV.

Two stage detector

A detector that does detection in two passes: first proposing candidate regions, then classifying each one. More accurate but slower than one stage detectors. Faster R-CNN and Mask R-CNN are canonical examples.

U-Net

A 2015 semantic segmentation architecture invented for biomedical imaging. Has an encoder bottleneck decoder shape (a U) with skip connections that preserve fine spatial details. Still widely used in 2026 in medical and scientific domains.

YOLO (You Only Look Once)

The most popular one stage detector lineage. Originally published in 2016, currently at YOLO11 (Ultralytics, 2024). The de facto standard for workforce object detection.

Zero shot segmentation

The ability to segment categories of objects that the model was never explicitly trained on, by using prompts (points, boxes, or text) instead of trained class labels. Segment Anything (SAM) is the canonical zero shot segmentation model.

References and Further Reading

Foundational Books

Szeliski, Richard. Computer Vision: Algorithms and Applications, Second Edition. Springer, 2022. Comprehensive coverage including detection and segmentation chapters. Available free online at szeliski.org/Book.

Elgendy, Mohamed. Deep Learning for Vision Systems. Manning Publications, 2020. Includes worked CV examples covering CNN backbones, transfer learning, and detection foundations that connect directly to this module.

Goodfellow, Ian; Bengio, Yoshua; and Courville, Aaron. Deep Learning. MIT Press, 2016. Free at deeplearningbook.org. The chapters on CNN architectures are the standard reference for the deep learning foundations under detection and segmentation.

Recent Books (2024 to 2026)

Antiga, Luca; Stevens, Eli; Huang, Howard; and Viehmann, Thomas. Deep Learning with PyTorch, Second Edition. Manning Publications, 2025 (ISBN 9781633438859). Hands on coverage of detection and segmentation models in PyTorch. Second edition adds transformers, diffusion models, and generative AI alongside the original CV foundations.

Ayyadevara, V Kishore and Reddy, Yeshwanth. Modern Computer Vision with PyTorch, Second Edition. Packt Publishing, June 2024 (ISBN 9781803231334). Comprehensive coverage of modern detection (YOLO, DETR), segmentation, transformers, foundation models, and production deployment. Particularly strong for this module.

Chollet, Francois and Watson, Matthew. Deep Learning with Python, Third Edition. Manning Publications, 2025 (ISBN 9781633436589). The Keras and TensorFlow perspective on detection and segmentation, accessible to engineers without heavy math. Includes dedicated chapters on image segmentation and object detection.

Jha, Ashish Ranjan. Mastering PyTorch, Second Edition. Packt Publishing, May 2024 (ISBN 9781801074308). Includes practical chapters on detection and segmentation with worked examples.

Foundational Papers

Redmon, Joseph; Divvala, Santosh; Girshick, Ross; and Farhadi, Ali. You Only Look Once: Unified, Real Time Object Detection. CVPR, 2016. The original YOLO paper.

Ren, Shaoqing; He, Kaiming; Girshick, Ross; and Sun, Jian. Faster R-CNN: Towards Real Time Object Detection with Region Proposal Networks. NIPS, 2015. The Faster R-CNN

paper that established the two stage detection pattern and introduced the Region Proposal Network.

He, Kaiming; Gkioxari, Georgia; Dollar, Piotr; and Girshick, Ross. Mask R-CNN. ICCV, 2017. The Mask R-CNN paper.

Ronneberger, Olaf; Fischer, Philipp; and Brox, Thomas. U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI, 2015. The original U-Net paper.

Carion, Nicolas et al. End to End Object Detection with Transformers. ECCV, 2020. The DETR paper that introduced transformer based detection.

Kirillov, Alexander et al. Segment Anything. ICCV, 2023. The original SAM paper, including the 1 billion mask dataset.

Ravi, Nikhila et al. SAM 2: Segment Anything in Images and Videos. Meta AI, 2024. The SAM 2 paper adding video segmentation.

Liu, Shilong et al. Grounding DINO: Marrying DINO with Grounded Pre Training for Open Set Object Detection. 2023. The GroundingDINO paper for open vocabulary detection.

Documentation and Tutorials

Ultralytics YOLO documentation at docs.ultralytics.com. The standard library for training YOLO models in 2026. Includes tutorials for custom dataset training that the lab uses.

Meta Segment Anything project at segment-anything.com. Includes the demo, paper, code, and integration guides for both SAM and SAM 2.

Hugging Face Transformers documentation at huggingface.co/docs. Hosts vision models including DETR variants and many segmentation models.

`segmentation_models_pytorch` (smp) on GitHub. Popular PyTorch library wrapping U-Net and variants with pretrained encoders.

Looking Ahead

Module 07 takes a step in a different direction. The first six modules have taught you how to extract information from images: what is there, where it is, what shape it has. Module 07 covers generative models for computer vision, which create new images rather than analyze existing ones. You will meet the modern systems (Stable Diffusion, FLUX, and others) and learn how to use them as tools in CV pipelines.

Critically, Module 07 is taught from a user perspective, not a builder perspective. You will use pretrained generative models from Hugging Face. You will not train one from scratch. The applications include data augmentation for small datasets, image editing, super resolution, inpainting, and synthetic training data generation. The depth of the diffusion architecture and the math of generative training is covered in ITAI 2376; in this course we focus on what generative models give you as a CV engineer.

The midterm project (Version 1) is due at the end of Module 06 or during Module 07, depending on your term calendar. Half the semester is now behind you. The remaining modules cover increasingly powerful and current techniques: generative models in Module 07, vision transformers and vision language models in Module 08, AI agents for computer vision in Module 09, video analysis in Module 10, and optimization and deployment in Module 11. Each builds on what you have already learned.